



LAPORAN TUGAS AKHIR

NetChat

Aplikasi Chat Client-Server Real-Time dengan Verifikasi Email
(Mata Kuliah Pemrograman Jaringan)

Nama : Givandra Apriliano

NPM : 50422623

Kelas : 4IA06

Mata Kuliah : Pemrograman Jaringan

Dosen : [Nama Dosen]

Program Studi Teknik Informatika
Fakultas Teknologi Industri
Universitas Gunadarma
2026

Daftar Isi

1. Judul dan Deskripsi Aplikasi
2. Arsitektur Sistem
3. Flowchart / Diagram Komunikasi
4. Penjelasan Implementasi
5. Hasil Pengujian
6. Kesimpulan
7. Jawaban Pertanyaan

1. Judul dan Deskripsi Aplikasi

1.1 Judul

NetChat — Aplikasi Chat Client-Server Real-Time dengan Verifikasi Email.

1.2 Deskripsi

NetChat adalah aplikasi *chat* berbasis web yang menerapkan konsep **client-server**. Banyak *client* (browser) terhubung ke satu *server* pusat, lalu saling bertukar pesan teks dan berkas secara *real-time* di dalam sebuah *room*. Komunikasi memanfaatkan dua protokol jaringan sekaligus: **HTTP** untuk autentikasi dan pengunggahan berkas, serta **WebSocket** (melalui pustaka Socket.IO) untuk pertukaran pesan dua arah yang berlangsung terus-menerus.

Sebagai fitur keamanan, proses registrasi dilengkapi **verifikasi email (kode OTP)**. Ketika mendaftar, server mengirim kode enam digit ke alamat email pengguna; akun baru hanya dapat digunakan untuk masuk setelah kode tersebut diverifikasi.

1.3 Tujuan

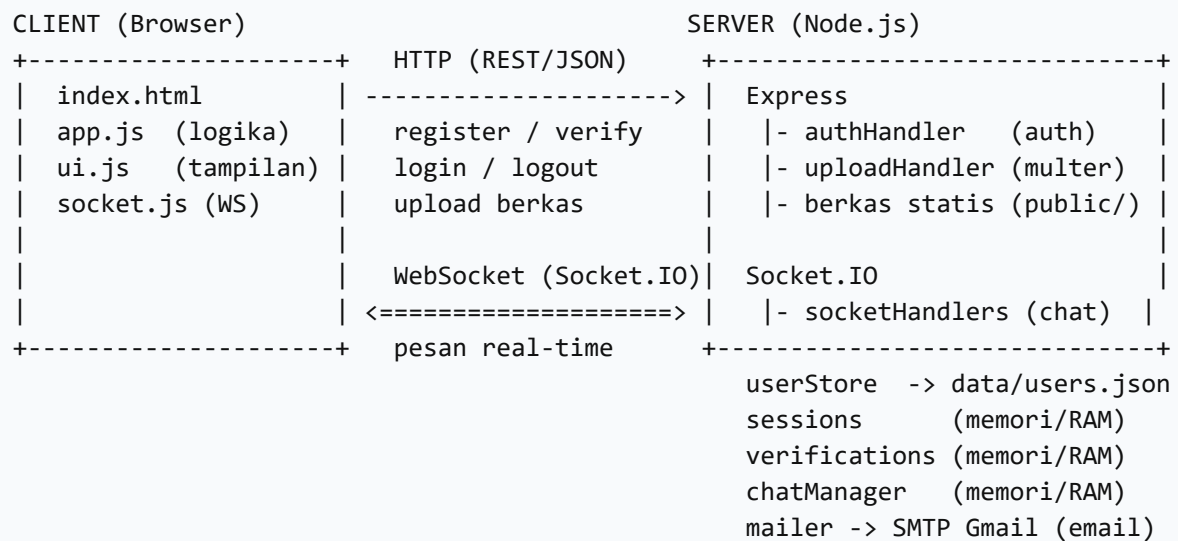
- Memahami dan menerapkan konsep komunikasi jaringan berbasis client-server.
- Mengimplementasikan komunikasi data melalui protokol HTTP dan WebSocket.
- Mengirim dan menerima data (pesan teks maupun berkas) melalui jaringan.
- Menangani koneksi, pengolahan pesan, dan penanganan kesalahan.
- Menambahkan lapisan keamanan berupa verifikasi email dan penyandian kata sandi.

1.4 Ruang Lingkup Fitur

- Registrasi dengan verifikasi kode email (OTP) melalui Gmail.
- Login berbasis token sesi (mendukung *auto-login* setelah *refresh*).
- Kata sandi disimpan sebagai *hash* scrypt + salt (bukan teks polos).
- Chat teks *real-time* di dalam room.
- Pengiriman berkas (gambar, PDF, dokumen Office, teks, zip) hingga 25 MB.
- Indikator "sedang mengetik", daftar anggota online, dan riwayat pesan.
- Penanganan kesalahan koneksi, validasi input, serta batas tipe/ukuran berkas.

2. Arsitektur Sistem

Aplikasi menganut model **client-server**. Sisi *client* berjalan di peramban dan bertanggung jawab atas tampilan serta interaksi pengguna. Sisi *server* berjalan di atas Node.js dan menangani autentikasi, penyimpanan data, penerusan pesan, serta pengunggahan berkas. Terdapat pula layanan luar, yaitu **SMTP Gmail**, untuk pengiriman email verifikasi.



Gambar 1. Diagram arsitektur sistem NetChat.

2.1 Komponen Utama

Lapisan	Komponen	Peran
Client	index.html, app.js, ui.js, socket.js	Antarmuka pengguna, logika autentikasi sisi client, render tampilan, dan koneksi WebSocket.
Server (HTTP)	Express + authHandler	Endpoint register, verify, resend, login, logout.
	uploadHandler (Multer)	Menerima unggahan berkas dan membalas metadata.
Server (WebSocket)	Socket.IO + socketHandlers	Event chat real-time: join, message, typing, disconnect.
Penyimpanan & layanan	userStore	Akun pengguna disimpan ke berkas data/users.json (hash script).
	sessions / verifications	Token sesi dan kode OTP di memori (RAM).
	chatManager	Daftar user per room dan riwayat pesan (RAM).

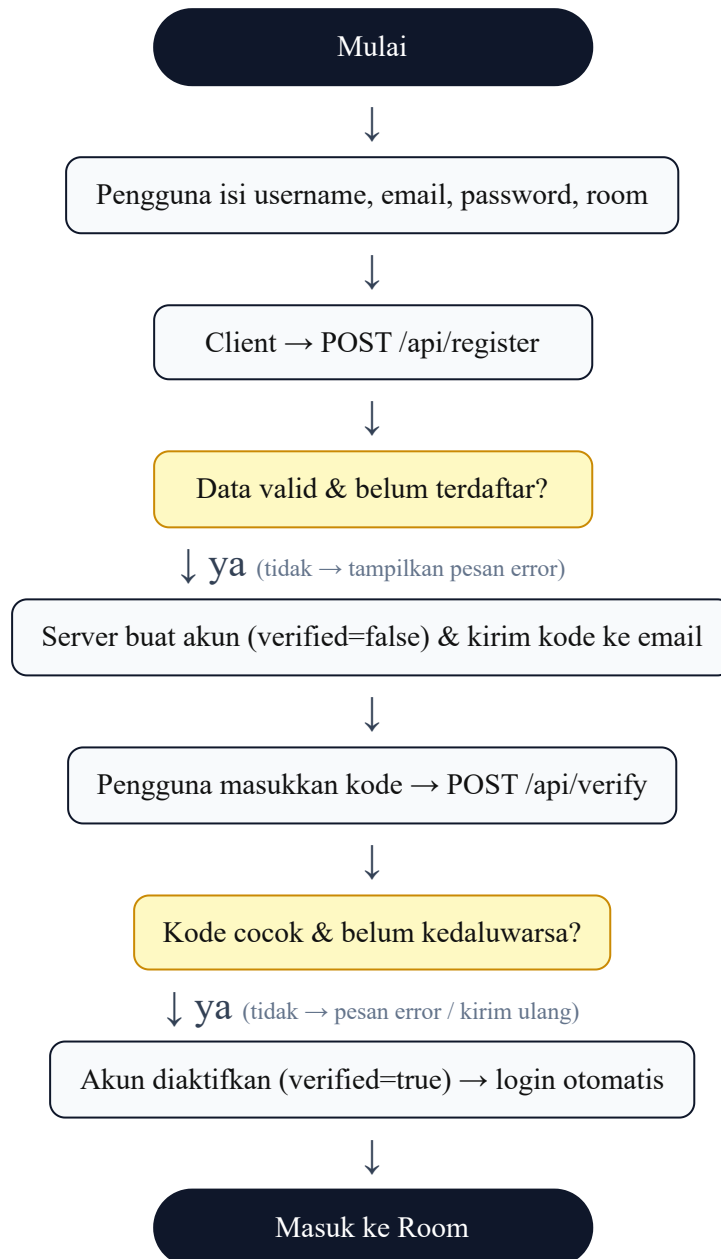
	mailer (Nodemailer)	Mengirim email kode verifikasi via SMTP Gmail.
--	---------------------	--

2.2 Teknologi yang Digunakan

Kebutuhan	Teknologi
<i>Runtime</i>	Node.js 18+
Server HTTP & routing	Express
Komunikasi real-time	Socket.IO (WebSocket)
Unggah berkas	Multer
Pengiriman email OTP	Nodemailer (SMTP Gmail)
Konfigurasi rahasia	dotenv (berkas .env)
Keamanan kata sandi	Modul crypto bawaan Node (scrypt)
Antarmuka	HTML, CSS (tema gelap ala shadcn/ui), ikon Lucide

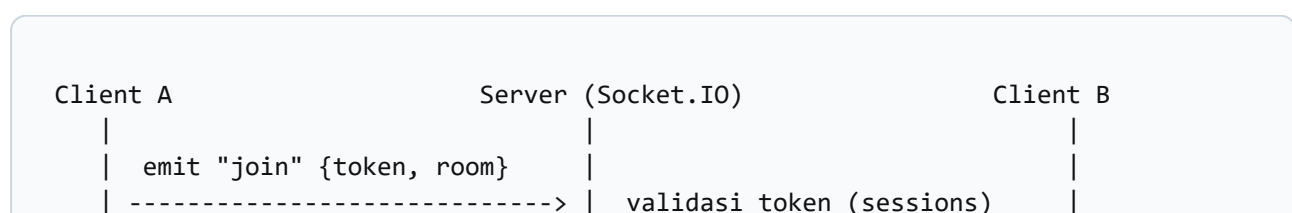
3. Flowchart / Diagram Komunikasi

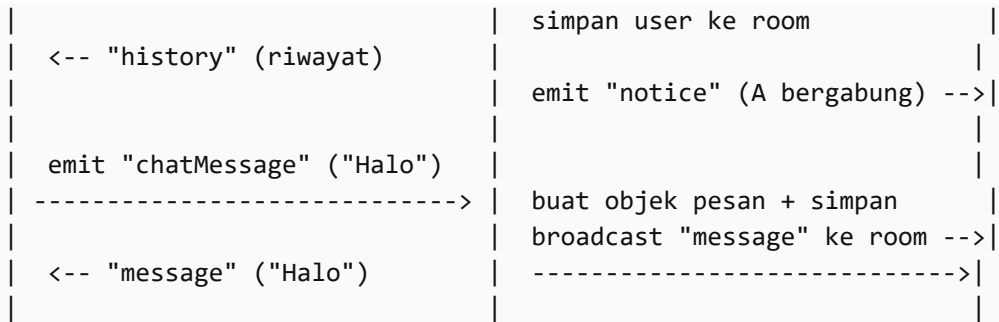
3.1 Alur Registrasi dan Verifikasi Email



Gambar 2. Flowchart alur registrasi dan verifikasi email.

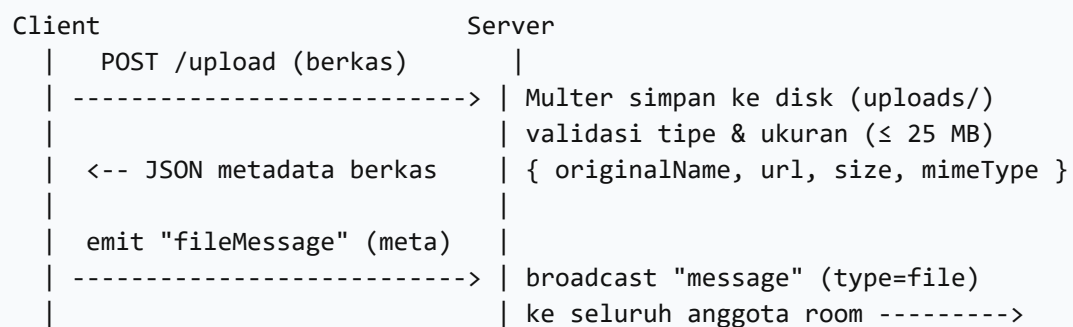
3.2 Diagram Komunikasi Chat (WebSocket)





Gambar 3. Diagram komunikasi pesan chat secara real-time.

3.3 Diagram Pengiriman Berkas (HTTP + WebSocket)



Gambar 4. Diagram pengiriman berkas (dipisah dari jalur chat teks).

4. Penjelasan Implementasi

4.1 Titik Masuk Server

Berkas `server.js` menggabungkan Express (HTTP) dan Socket.IO (WebSocket) di atas satu *HTTP server* yang sama. Express menyajikan berkas statis dari folder `public/`, menangani endpoint autentikasi dan unggah, sedangkan Socket.IO menangani setiap koneksi *real-time*.

```
const server = http.createServer(app);
const io = new Server(server);

app.use(express.json());
app.use(express.static(path.join(__dirname, "..", "public")));
app.use("/uploads", express.static(UPLOAD_DIR));
app.use("/", authHandler);    // /api/register, /api/verify, /api/login, ...
app.use("/", uploadHandler);  // /upload

io.on("connection", (socket) => registerSocketHandlers(io, socket));
server.listen(PORT);
```

4.2 Keamanan Kata Sandi (scrypt + salt)

Kata sandi tidak pernah disimpan dalam bentuk teks. Saat registrasi, sistem menghasilkan *salt* acak dan menyimpan *hash* scrypt. Saat login, perbandingan dilakukan dengan `timingSafeEqual` untuk mencegah *timing attack*.

```
function hashPassword(password) {
  const salt = crypto.randomBytes(16).toString("hex");
  const hash = crypto.scryptSync(password, salt, 64).toString("hex");
  return { salt, hash };
}
```

4.3 Verifikasi Email (OTP)

Modul `verifications.js` membuat kode enam digit yang berlaku 10 menit dan maksimum 5 kali percobaan. Modul `mailer.js` mengirimnya via Nodemailer. Bila kredensial SMTP belum diatur, kode dicetak ke *console* (mode pengembangan) sehingga aplikasi tetap dapat diuji.

```
// verifications.js – membuat kode OTP
function generateCode() {
  const num = crypto.randomInt(0, 10 ** VERIFICATION.CODE_LENGTH);
```



```
return String(num).padStart(VERIFICATION.CODE_LENGTH, "0");
}
```

Login ditolak apabila akun belum terverifikasi:

```
if (!record.verified) {
  return { success: false, needVerify: true,
    error: "Email belum diverifikasi..." };
}
```

4.4 Identitas Berbasis Token pada WebSocket

Ketika bergabung ke room, identitas pengguna diambil dari **token sesi**, bukan dari input mentah client. Dengan demikian hanya pengguna yang sudah login yang dapat masuk room.

```
socket.on("join", ({ token, room }, callback) => {
  const username = sessions.getUser(token);
  if (!username) return callback?({ success: false, authError: true });
  // ... user ditambahkan ke room, riwayat dikirim, anggota diberi tahu
});
```

4.5 Pemisahan Transfer Berkas

Berkas dikirim lewat HTTP (Multer) lalu *metadata*-nya disebarkan lewat WebSocket. Pemisahan ini menjaga jalur WebSocket tetap ringan dan sekaligus mendemonstrasikan penggunaan dua protokol. Multer memvalidasi tipe MIME dan membatasi ukuran berkas maksimum 25 MB.

4.6 Penanganan Kesalahan (Error Handling)

- **Validasi input:** panjang username/password, format email, nama room wajib diisi.
- **Autentikasi:** balasan HTTP 401 (kredensial salah) dan 403 (belum verifikasi).
- **Unggah berkas:** pesan khusus untuk melebihi batas ukuran atau tipe tak didukung.
- **Koneksi WebSocket:** token tidak valid memicu permintaan login ulang; *disconnect* memberi tahu anggota room dan memperbarui daftar online.
- **Tingkat proses:** `process.on("uncaughtException")` mencegah server mati tanpa jejak; berkas `users.json` yang rusak ditangani dengan aman.
- **Sisi client:** setiap permintaan dibungkus try/catch dengan pesan "Tidak dapat terhubung ke server."

5. Hasil Pengujian

Pengujian dilakukan dengan menjalankan server secara lokal (<http://localhost:3000>) dan membuka aplikasi pada dua proses/perangkat berbeda untuk mensimulasikan komunikasi antar-client.

5.1 Skenario dan Hasil

No	Skenario Pengujian	Hasil Diharapkan	Status
1	Registrasi akun baru (username, email, password)	Akun dibuat, kode OTP terkirim ke email	Berhasil
2	Login sebelum verifikasi email	Ditolak (HTTP 403, minta verifikasi)	Berhasil
3	Verifikasi dengan kode salah	Ditolak (HTTP 400, sisa percobaan berkurang)	Berhasil
4	Verifikasi dengan kode benar	Akun aktif, login otomatis ke room	Berhasil
5	Login setelah verifikasi	Berhasil, memperoleh token sesi	Berhasil
6	Kirim pesan teks antar dua client	Pesan diterima seketika (real-time)	Berhasil
7	Kirim berkas (PDF/gambar)	Berkas terkirim & dapat diunduh anggota room	Berhasil
8	Indikator "sedang mengetik"	Tampil pada anggota lain	Berhasil
9	Anggota keluar / koneksi terputus	Notifikasi keluar & daftar online diperbarui	Berhasil
10	Unggah berkas melebihi 25 MB / tipe tak didukung	Ditolak dengan pesan kesalahan	Berhasil

5.2 Tangkapan Layar (Screenshot)

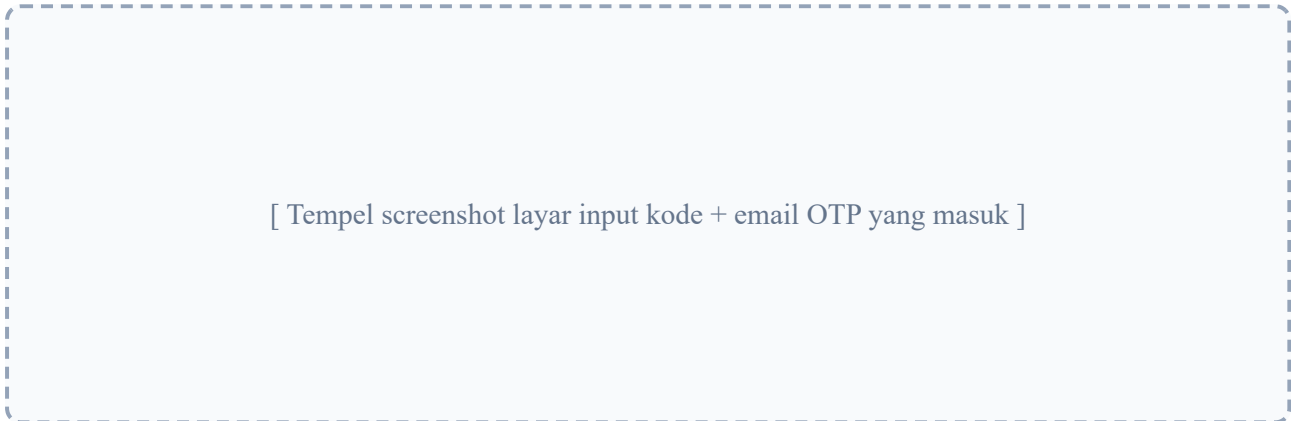
a. Tampilan aplikasi — Halaman Login & Registrasi

[Tempel screenshot halaman login/registrasi (UI terbaru) di sini]



Gambar 5. Tampilan halaman login dan registrasi.

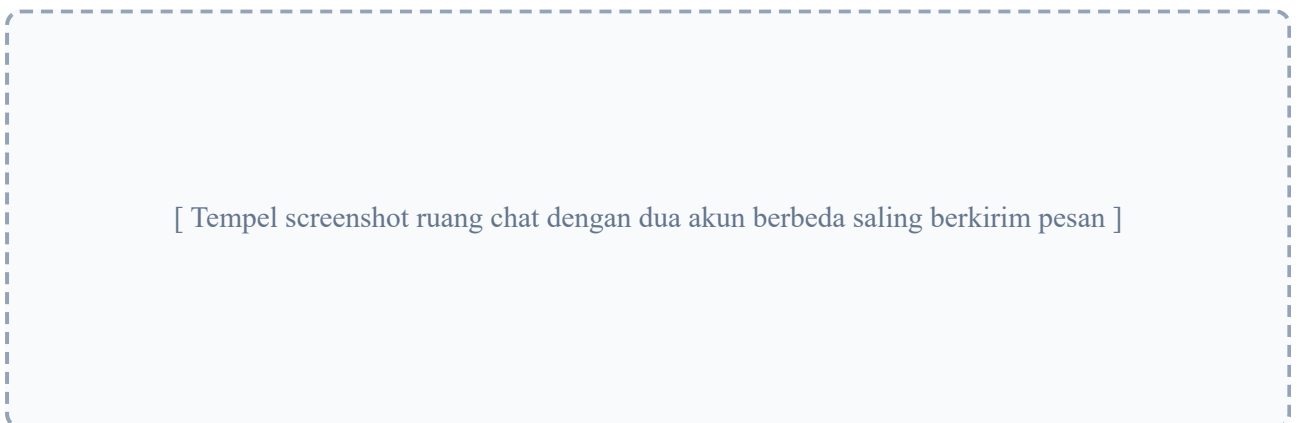
b. Verifikasi email — Layar kode & email yang diterima



[Tempel screenshot layar input kode + email OTP yang masuk]

Gambar 6. Proses verifikasi email (kode OTP).

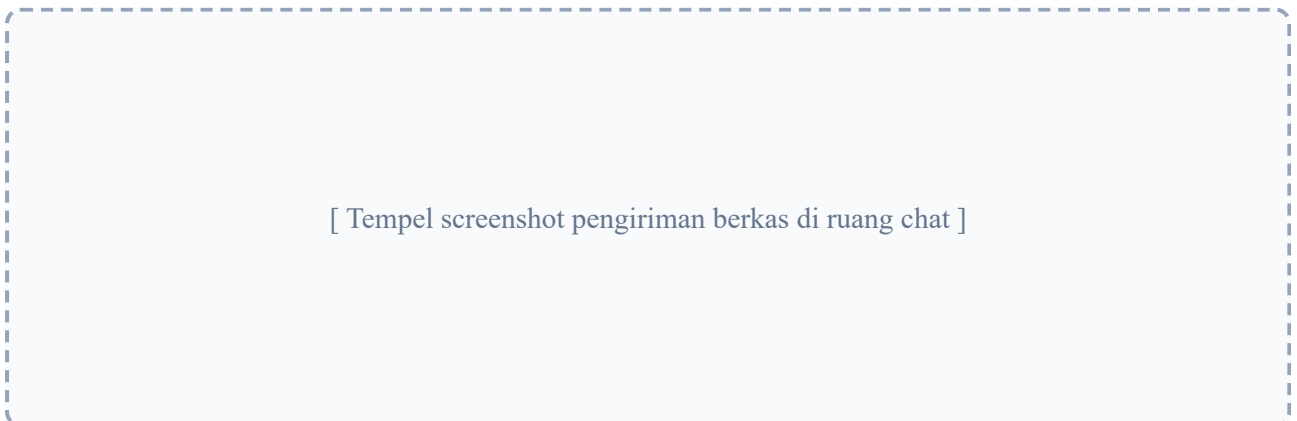
c. Proses komunikasi antara client dan server — Ruang Chat



[Tempel screenshot ruang chat dengan dua akun berbeda saling berkirim pesan]

Gambar 7. Komunikasi pesan real-time antar dua client dalam satu room.

d. Hasil pengujian — Pengiriman berkas



[Tempel screenshot pengiriman berkas di ruang chat]

Gambar 8. Pengujian fitur pengiriman berkas.

e. Sisi server — Log komunikasi di terminal

[Tempel screenshot terminal server (log connect/disconnect atau kode OTP)]

Gambar 9. Log aktivitas pada sisi server.

Catatan: ganti setiap kotak putus-putus di atas dengan tangkapan layar Anda. Pada berkas HTML, ganti baris `<div class="ss">...</div>` menjadi ``, lalu cetak ulang ke PDF (Ctrl+P → Save as PDF).

6. Kesimpulan

Aplikasi NetChat berhasil mengimplementasikan konsep komunikasi jaringan berbasis **client-server** menggunakan Node.js. Aplikasi memanfaatkan dua protokol jaringan, yaitu **HTTP** untuk autentikasi dan pengunggahan berkas serta **WebSocket** untuk pertukaran pesan *real-time*. Seluruh tujuan tugas tercapai: data dapat dikirim dan diterima antar-proses, koneksi dan pesan dikelola dengan baik, serta kesalahan ditangani secara memadai.

Nilai tambah aplikasi ini adalah lapisan keamanan berupa **verifikasi email (OTP)** saat registrasi dan penyandian kata sandi dengan **scrypt + salt**. Berdasarkan sepuluh skenario pengujian, seluruh fungsi berjalan sesuai harapan.

Keterbatasan. Sesi login dan riwayat pesan disimpan di memori sehingga hilang saat server dimulai ulang; data pengguna disimpan pada berkas JSON, bukan basis data. Aplikasi juga belum menerapkan pembatasan laju (*rate limiting*) dan enkripsi *end-to-end*.

Pengembangan selanjutnya. Penggunaan basis data (mis. PostgreSQL atau MongoDB), HTTPS/TLS, pembatasan laju untuk mencegah *brute-force*, penyimpanan berkas ke layanan objek, serta *deployment* publik (mis. melalui Cloudflare Tunnel) dapat meningkatkan keamanan, kecepatan, dan keandalan aplikasi.

7. Jawaban Pertanyaan

7.1 Protokol jaringan apa yang digunakan pada aplikasi Anda? Jelaskan alasannya.

Aplikasi menggunakan **dua protokol yang keduanya berjalan di atas TCP**:

- **HTTP** — untuk menyajikan halaman web, autentikasi (register, verify, login, logout), dan pengunggahan berkas. HTTP dipilih karena bersifat *request-response* yang cocok untuk operasi sekali jalan seperti login dan unggah berkas, serta mudah diamankan dengan HTTPS.
- **WebSocket** (melalui Socket.IO) — untuk chat *real-time*. WebSocket dipilih karena menyediakan koneksi **dua arah yang tetap terbuka (*full-duplex*)**, sehingga server dapat langsung mendorong pesan ke client tanpa perlu *polling* berulang. Ini membuat pesan, notifikasi, dan indikator mengetik tampil seketika dengan overhead rendah.

TCP dipilih (bukan UDP) karena chat menuntut **keandalan dan urutan pesan** yang terjamin — tidak boleh ada pesan yang hilang atau tertukar urutannya.

7.2 Bagaimana mekanisme komunikasi antara client dan server?

Client pertama-tama melakukan autentikasi via HTTP. Setelah login, server memberi **token sesi**. Client kemudian membuka koneksi **WebSocket** dan mengirim event join beserta token tersebut; server memvalidasi token, memasukkan client ke room, lalu mengirimkan riwayat pesan. Selanjutnya:

- Client mengirim event `chatMessage`; server membuat objek pesan, menyimpannya, lalu *broadcast* event message ke seluruh anggota room.
- Untuk berkas, client mengunggah via HTTP POST `/upload`, menerima metadata, lalu mengirim event `fileMessage` berisi metadata tersebut untuk disebarkan ke room.
- Event pendukung seperti `typing/stopTyping` dan `roomUsers` menjaga pengalaman tetap interaktif.

Identitas pengirim selalu diambil dari token/pemetaan socket di server, bukan dari data mentah client, demi keamanan.

7.3 Bagaimana aplikasi menangani kesalahan koneksi atau kegagalan pengiriman data?

- **Kegagalan permintaan HTTP** dibungkus try/catch di sisi client dengan pesan yang jelas kepada pengguna.

- **Token tidak valid / sesi berakhir** (mis. server restart) terdeteksi saat join; client diarahkan untuk login ulang.
- **Reconnect otomatis:** Socket.IO mencoba menyambung kembali ketika koneksi terputus; indikator status ("Terhubung/Terputus") memberi umpan balik.
- **Kesalahan unggah** (ukuran > 25 MB atau tipe tak didukung) dibalas dengan pesan spesifik, bukan sekadar gagal diam.
- **Ketahanan server:** `uncaughtException` dicegat agar server tidak mati mendadak, dan berkas data yang rusak ditangani aman.
- **Kode OTP** memiliki masa berlaku dan batas percobaan, dengan opsi "kirim ulang kode" bila gagal.

7.4 Apa kelebihan dan kekurangan implementasi aplikasi Anda?

Kelebihan:

- Menggunakan dua protokol (HTTP + WebSocket) sehingga tiap kebutuhan ditangani protokol yang paling sesuai.
- Komunikasi *real-time* dengan latensi rendah tanpa *polling*.
- Keamanan: verifikasi email OTP, kata sandi ter-*hash* (scrypt+salt), identitas berbasis token, dan pencegahan XSS pada tampilan pesan.
- Kode modular dan terpisah berdasarkan tanggung jawab, mudah dipelihara.
- Ringan — tidak memerlukan basis data untuk dijalankan.

Kekurangan:

- Sesi dan riwayat pesan disimpan di memori sehingga hilang saat server dimulai ulang.
- Data pengguna pada berkas JSON kurang ideal untuk skala besar.
- Belum ada HTTPS/TLS bawaan, *rate limiting*, maupun enkripsi *end-to-end*.
- Belum mendukung banyak instance server (penskalaan horizontal).

7.5 Pengembangan agar aplikasi lebih aman, cepat, dan andal.

Lebih aman: menerapkan HTTPS/TLS, pembatasan laju (*rate limiting*) pada login dan pengiriman kode untuk mencegah *brute-force*, validasi & sanitasi input yang lebih ketat, enkripsi *end-to-end*, serta manajemen sesi dengan masa berlaku (*expiry*) dan penyegaran token.

Lebih cepat: memanfaatkan *caching*, kompresi, dan CDN untuk aset statis; mengoptimalkan/mengecilkan ukuran berkas media; serta menyajikan berkas unggahan melalui penyimpanan objek yang tersebar secara geografis.

Lebih andal: memakai basis data persisten (PostgreSQL/MongoDB) untuk pesan dan sesi, penskalaan horizontal beberapa instance server dengan *adapter* Socket.IO (mis. Redis), pemantauan/*logging*, mekanisme *backup*, serta *deployment* publik yang stabil (mis. melalui Cloudflare Tunnel atau layanan *hosting* Node).